Application of Stack

Infix (operand → operator)	Post	Prefix
1+2	12+	+12
1-2	12-	-12
2^1	21^	^21

Associativity

2^3^2 → 2^9 → 512

8^2 → 64

5-2-2 → 3-2 → 1

5-3-2 → 5-5 → 0

( )	→ N/A
^	→ R to L
/*	→ L to R
+-	→ L to R

↑ heighest

↓ lowest

Calculator:

$$\frac{3*4}{6+\frac{3}{4}}$$

$= 1.777\ldots$

Inf. to Post F

$(3*4)/(6+(3\div4))$

$= (34*)/(6(34\div)+)$

$= (34*)(6(34\div)+)/$

$= 34*634\div+/$

with stack

Push(3) → [3]

Push(4) → [4, 3]

multi → [12]

Push(6) → [4, 3, 6]

Push(3) → [3, 6, 12]

Push(4) → [4, 3, 6, 12]

DIV() → [0.75, 6, 12] (3÷4)

ADD() → [6.75, 12] → DIV() → [1.75]

3 condition

(i) Check stack condition — (1)

(ii) Check if scanned character is a bracket — (2,3)

(iii) if the scanned character is an operator — (4,5,6)

9+2-5

(*) Q:) 2^3^4+5*7

Scanned i/P	Stack	Output
2		2
^	^	2
3	^	23
^	^^	23
4	^^	234
+	^	234^
+	+	234^^
5	+	234^^5
*	*+	234^^5
7	*+	234^^57

Power কম হলে নিজে না ঢুকে অন্য ও operator কে বের করে দিবে যতক্ষন নিজে না ঢুকতে পারে

↑ this two

Ans → 234^^57*+

© 1+3^2+(5*6-4)*7

Scanned i/p	stack	output
1		1
+	+	1
3	+	13
^	^+	13
2	^+	132
+	+	132^
+	+	132^+
(	(+	132^+
5	(+	132^+5
*	*(+	132^+5
6	*(+	132^+56
-	(+	132^+56*
-	-(+	132^+56*
4	-(+	132^+56*4
)	(+	132^+56*4-
*	*(+	132^+56*4-
7	*(+	132^+56*4-7
		132^+56*4-7*+

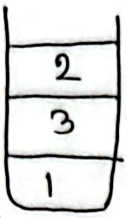
')' এটা আসলে stack থেকে একটা element বাদ দেয়

')' এটা output এ যাবে না

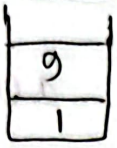
25-07-2023

* 132^+56*4-7*+ with Stack-

Push(1)
Push(3)
Push(2)

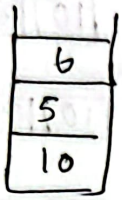


Power (3^2)

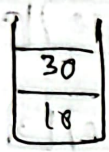


Add( )

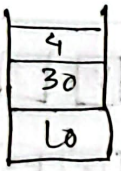
Push(5)
Push(6)



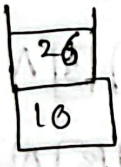
multi



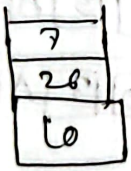
Push(4)



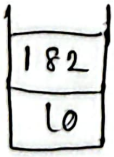
Sub



Push(7)



multi



Add( )

* 2 or more digit.

Q. - $10 \wedge 3 + (26 * 6) = 1156$

I/p	Stack	output
1	⊐	1
0	⊐	10
^	^	10
3	^	10 3
+	+	10 3 ^
(	(+	10 3 ^
2	(+	10 3 ^ 2
6	(+	10 3 ^ 26
*	*(+	10 3 ^ 26
6	*(+	10 3 ^ 26 6
)	(+	10 3 ^ 26 6 *
		10 3 ^ 26 6 * +

. Aim

now,

Push(10) | 3 |
Push(3) | 10 |

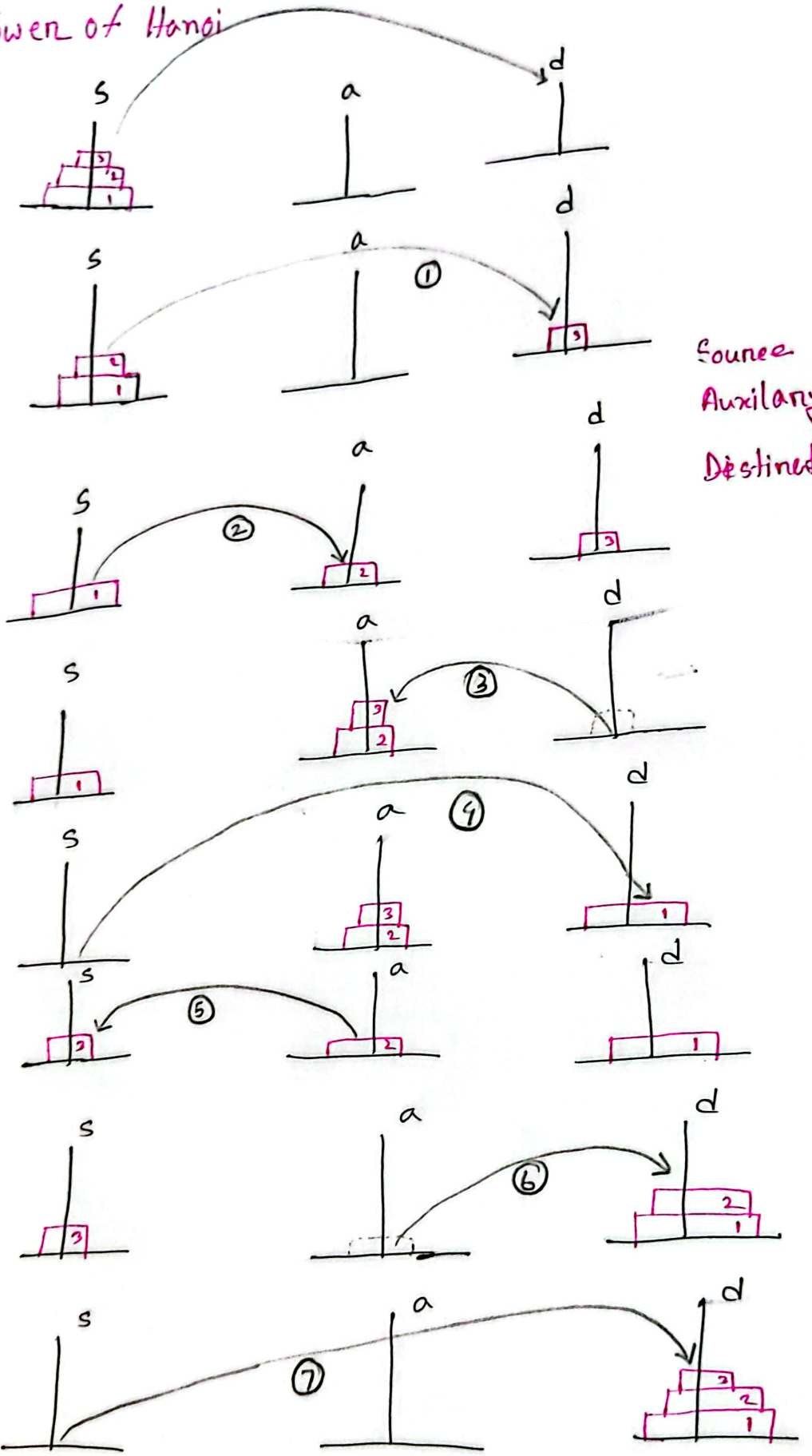
Power() | 10^3 |

Push(26) | 26 | 10^3 |

Push(6) | 6 | 26 | 1000 | → multi() | 156 | 1000 | → add() | 1156 | . Proved.

Recursion

Tower of Hanoi



Source — s

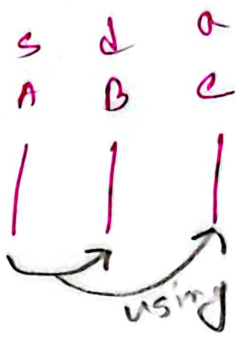
Auxilary — a

Destination — d

① এর এর সমান হবে

② Pressure থাকে না হয়

③ $2^n - 1$ moves হবে



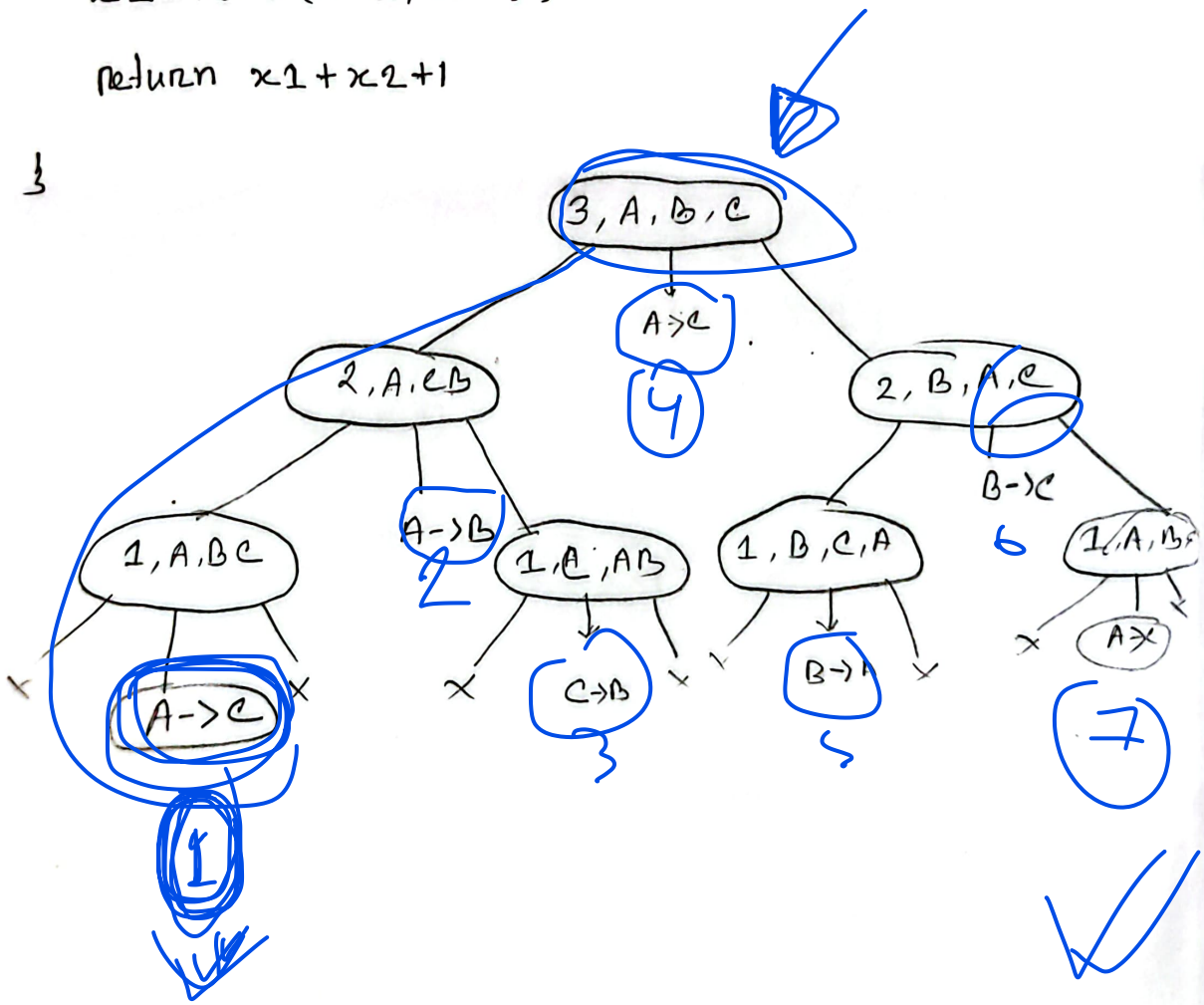
Function

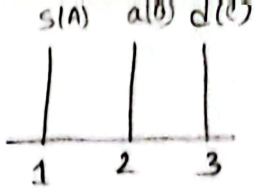
```
int Hanoi (int n, int s, int d, int a) {
  if (n==1) {
      printf ("%d -> %d", s, d);
      return 1;
  }
  else {
     x1=Hanoi (n-1, s, a, d);
     print ("%d -> %d", s, d); ~~count++~~;
     x2= Hanoi (n-1, a, d, s);
     return x1+x2+1
  }
}
```

A =

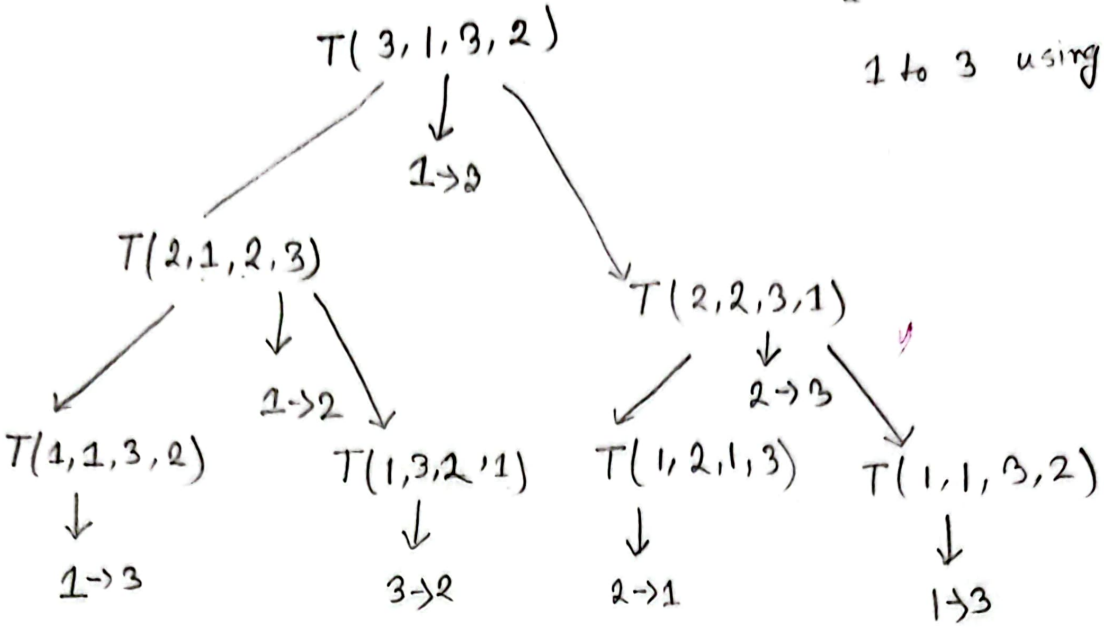
B =

C =

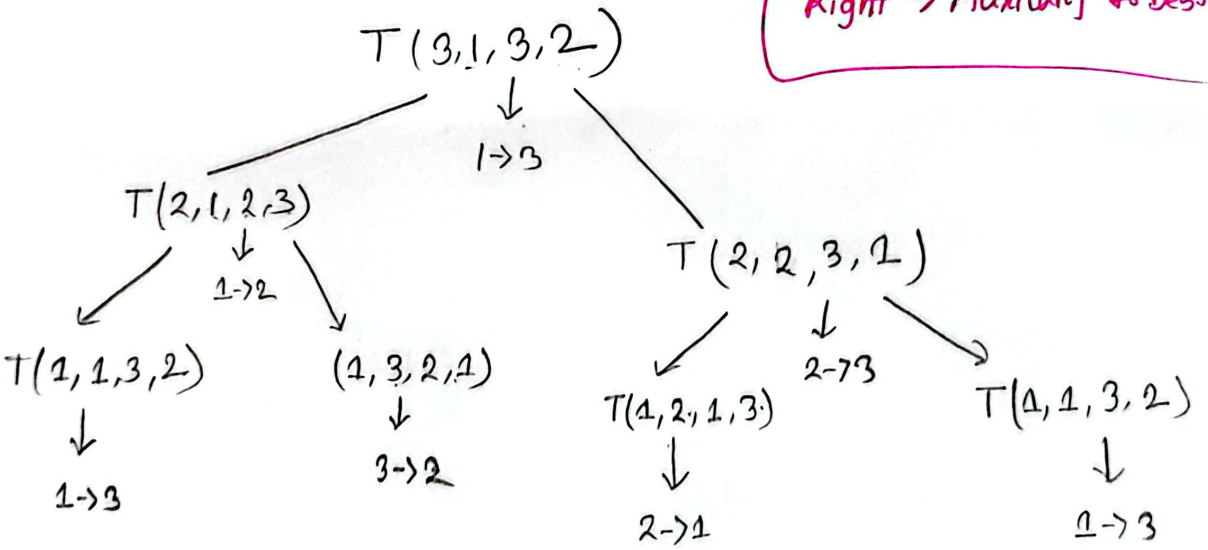




1 to 3 using 2

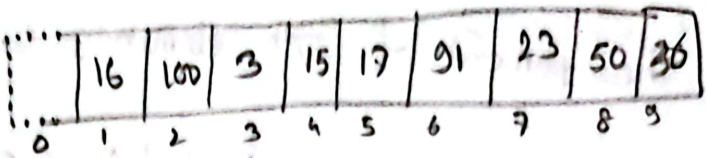


Left → Source to Auxi
Right → Auxilary to Dest



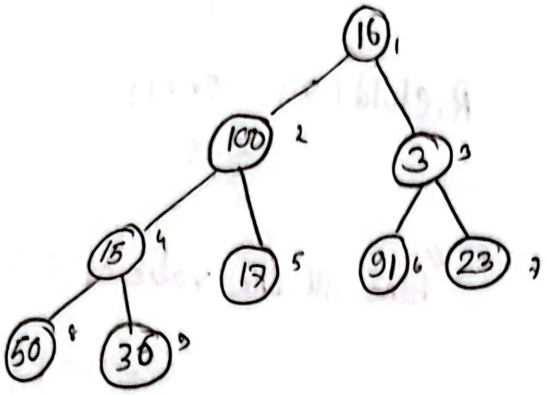
1->3 1->2 3->2 1->3 2->1 2->3 1->3

Binary Heap



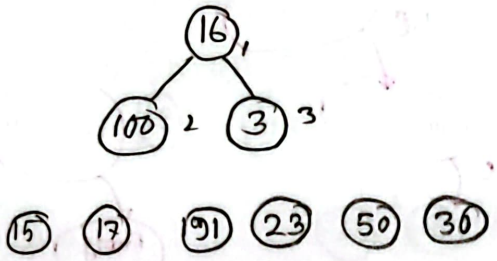
length – 9 [indexing ⇒ 1-9]

heap size – 9

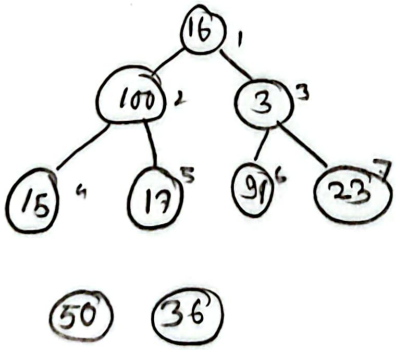


heap size এর value বারালে element serial এর বাইরে চলে যাবে

heap size (2) '-'



heap size (7) '+'



Given array থেকে Parent কি/ কোন Index এ আছে, /child → left, Right

Who is the parent of 7th element? ↵

Parent (i) = $\lfloor \frac{i}{2} \rfloor$

Parent (7) = $\lfloor \frac{7}{2} \rfloor$

= $\lfloor 3.5 \rfloor$

= 3.

① L.child (7) = 2*7 left child ≤ heapSize

= 14

R.child (7) = 2*7+1

= 15

"here all the value of (i) is index value"

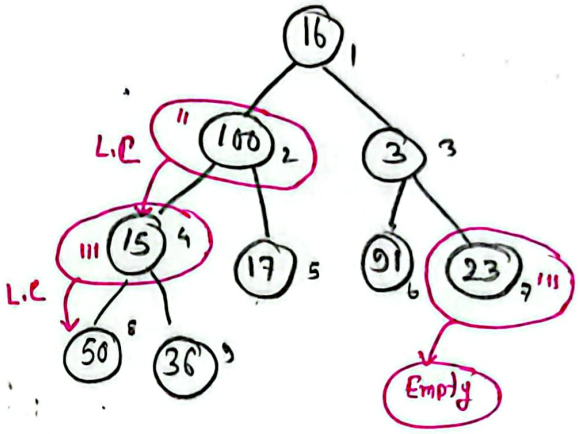
(ii) left chied(i) = 2*i

left chield (2) = 2*2 = 4

Right child (i) = 2i+1

Right child (2) = 2*2+1

= 5



(iii) left child (4) = 2*4 = 8

Max heap

Parent ≥ Children [Parent Child থেকে বড় হলে Max heap]

16 14 10 8 7 9 3 2 1 4

P

Min heap

Parent ≤ Children

3 4 10 8 7 19 13 12 14 16

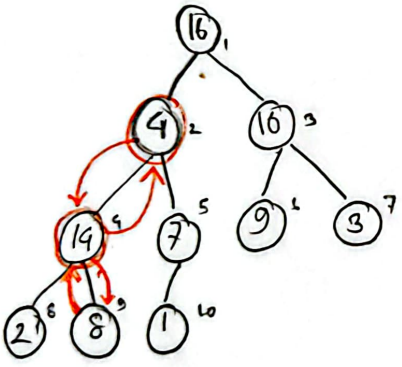
P

Max Heapify (O(log n))

↳ try to make array max

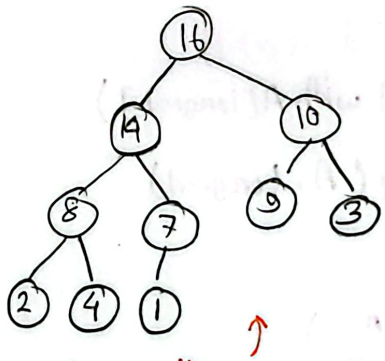
Build Max Heapify (O(n))

↳ Convert an array in to a heap.



maxHeapify(A,2)

maxHeapify(A,4)



after calling maxHeapify() Function

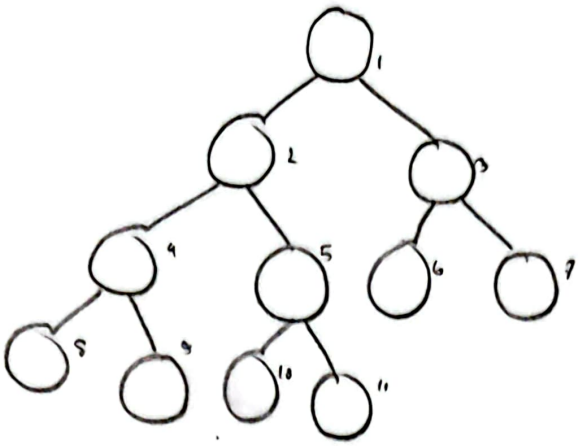
Code

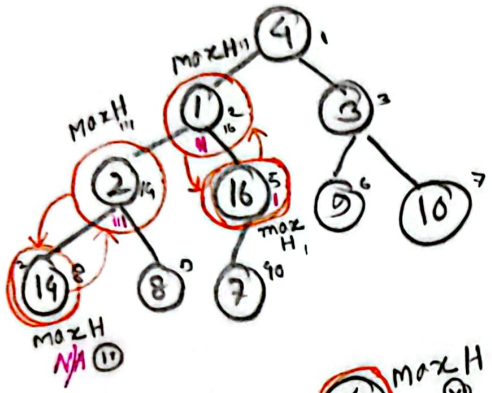
```
maxHeapify(A,i)
    left = 2*i;
    Right = 2*i + 1;
if (left <= heapSize && A[i] >= A[left]
    largest = i;
else
    laregest = left
if (Right <= heapSize && A[Right] > A[Largest])
    Largest = right
if i != Largest
    swap(A[i] with A[Largest])
    maxHeapify(A, Largest)
```

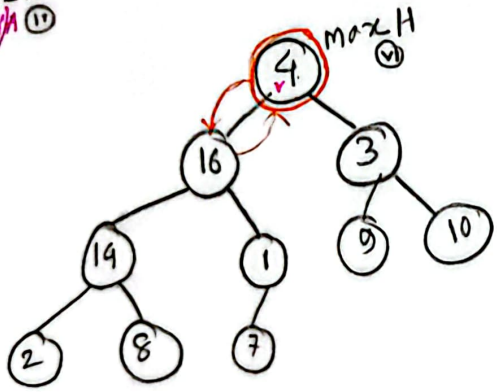
uild MaxHeapify(A)

```
A.heapSize = A.length
for i = ⌊A.Length/2⌋ down to 1
    MaxHeapify(A,i)
```

Build Max Heapify → Convirt an array into a Heap.







last index কে 2 দিয়ে ভাগ দিলে যে parent
আসবে সেটার ceiling value নিবো এবং যদি
উনি যে parent এর children আছেন তাদের
মধ্যে index থেকে main root পর্যন্ত maxHeapify
function call হবে

$$A(i) = \left\lfloor \frac{10}{2} \right\rfloor = \lfloor 5 \rfloor = 5$$

Ch-3

Heap Sort

HeapSort (A)

1. Build_Max_Heapify (A)
2. for(i = A.length. down to 2)
3. swap A[1] with A[i]
4. A.heapSize = A.heapSize - 1
5. MaxHeapify (A,1)

16, 14, 10, 8, 7, 9, 3, 2, 4, 1